Tyr-Crypto
Computational Handbook

Volume 1: ChaCha20 and XChaCha20

A guided calculation book for addition, XOR, rotations, quarter-rounds, state rounds, counters, HChaCha20, and XChaCha20.

Assumed knowledge: only Volume 0. Every method below contains at least four worked examples before the exercises.

Blue	new idea, input, or plain-language explanation
Purple	exact recipe to follow
Green	fully worked calculation
Orange	exercise for the reader
Red	misuse, security, or implementation danger
Teal	checkpoint before continuing

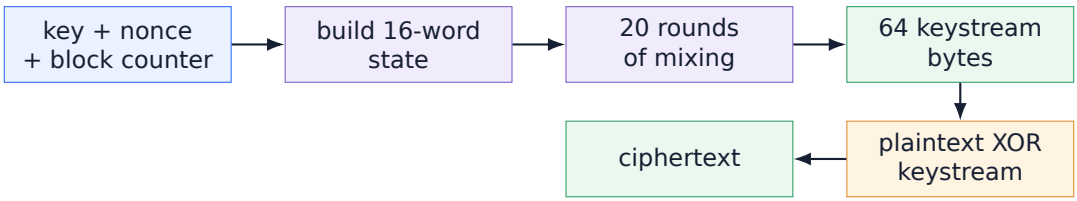
Repository snapshot reviewed: commit `fb9cf27926140e95c02362d4fe1c349ff65d3dd0`, July 2026.
Square-corner teaching edition.

Contents

1 The entire algorithm before the details

Plain-language explanation

ChaCha20 does not move the plaintext directly through a complicated encryption formula. It first creates a long row of apparently random bytes called a **keystream**. The plaintext is then XORed with that keystream.



Do not skip this warning

ChaCha20 and XChaCha20 are stream ciphers. They hide data but do not detect modification. A message authentication method such as Poly1305 is needed for integrity. Volume 2 explains that composition.

1.1 The four objects that must stay separate

Object	Plain meaning	Size in Tyr-Crypto
key	Secret selector for the keystream.	32 bytes
nonce	Public per-message value. It must not repeat with the same key.	12 bytes for ChaCha20; 24 for XChaCha20
counter	Which 64-byte keystream block is being generated.	32-bit unsigned integer
state	Sixteen 32-bit working words arranged as a 4 by 4 grid.	64 bytes

2 Method 1: read four bytes as one little-endian word

Input: four bytes
Do: reverse their visual order when writing the word
Output: one 32-bit word

Plain-language explanation

The machine sees bytes in increasing memory addresses. Little-endian means the first byte supplies the smallest place value. The byte row `12 34 56 78` therefore represents `0x78563412`, not `0x12345678`.

Mechanical rule

For bytes b_0, b_1, b_2, b_3:

$$W = b_0 + 2^8 b_1 + 2^{16} b_2 + 2^{24} b_3.$$

A quick paper method is: write the four two-digit byte groups in reverse order.

Worked example 1 - four unrelated bytes

`12 34 56 78` → reverse the byte groups → `0x78563412`.

Worked example 2 - increasing key bytes

`00 01 02 03` → `0x03020100`. This is the first key word in the RFC test vector.

Worked example 3 - leading zeros are part of the word

`09 00 00 00` → `0x00000009`. Do not throw the zeros away before placing the byte groups.

Worked example 4 - all high bytes

`FF 80 01 A0` → `0xA00180FF`.

Exercises

Convert to 32-bit little-endian words: (a) `DE AD BE EF`, (b) `10 11 12 13`, (c) `00 00 4A 00`, (d) `65 78 70 61`.

Checkpoint

You are ready to continue when you can explain why `65 78 70 61` becomes `0x61707865`.

3 Method 2: add 32-bit words with wrap-around

Input: two 32-bit words
Do: ordinary addition, then keep the lowest 32 bits
Output: one 32-bit word

Plain-language explanation

A 32-bit word has room for numbers from 0 to $2^{32}-1$. When an addition reaches 2^{32}, the extra carry falls off the left edge. This is the same idea as a car odometer returning to zero.

Mechanical rule

$$a + b \text{ in ChaCha} = (a + b) \bmod 2^{32}.$$

In hexadecimal, keep exactly eight digits on the right.

Worked example 1

`0x00000001` + `0x00000002` = `0x3`. Keep the lowest eight hexadecimal digits: `0x00000003`.

Worked example 2

`0xFFFFFFFF` + `0x00000001` = `0x100000000`. Keep the lowest eight hexadecimal digits: `0x00000000`.

Worked example 3

`0xFFFFFFFE` + `0x00000005` = `0x100000003`. Keep the lowest eight hexadecimal digits: `0x00000003`.

Worked example 4

`0x80000000` + `0x80000000` = `0x100000000`. Keep the lowest eight hexadecimal digits: `0x00000000`.

Exercises

Compute with 32-bit wrap-around: (a) `0x00000009+0x00000007`, (b) `0xFFFFFFFF+0x00000002`, (c) `0xFFFFFF00+0x00000200`, (d) `0x7FFFFFFF+0x00000001`.

4 Method 3: XOR two words

Input: two equally wide bit rows
Do: write 1 where the bits differ
Output: one mixed word

Mechanical rule

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

XOR is reversible: $(P \oplus K) \oplus K = P$.

Worked example 1 - one nibble

$1010 \oplus 1100 = \boxed{0110}$, so 0xA XOR 0xC = 0x6.

Worked example 2 - one byte

10110110 XOR 01011001 = 11101111 = 0xEF.

Worked example 3 - same word

0x12345678 $\oplus$ 0x12345678 $=$ 0x00000000 because every compared bit is equal.

Worked example 4 - XOR twice cancels

Let P = 0x0000002A and K = 0x000000F0. Then $C = P \oplus K$ = 0x000000DA and $C \oplus K$ = 0x0000002A.

Exercises

Compute: (a) 0x55 XOR 0x0F, (b) 0xAAAAAAAA XOR 0x55555555, (c) 0xDEADBEEF XOR itself, (d) recover P from C = 0x93 and K = 0xA5.

5 Method 4: rotate a fixed-width word left

Input: one word and a distance
Do: move left; return escaped bits on the right
Output: same bits in a new order

Plain-language explanation

A rotation never loses bits. A left shift does lose bits. Draw the word as a loop when you feel unsure.

Mechanical rule

For a w-bit word:

$$\text{rotl}_w(x, n) = ((x \ll n) \bmod 2^w) \text{ OR } (x \gg (w - n)).$$

Worked example 1 - 8-bit rotation by 1

`10010110` $\xrightarrow{\text{rotl}_8(1)}$ `00101101`. The leftmost 1 returns on the right.

Worked example 2 - 8-bit rotation by 4

`0xAB=1010 1011`. Rotating by four swaps the two nibbles: `0xBA`.

Worked example 3 - rotation by the word width

$\text{rotl}_{32}(\texttt{0x12345678}, 32) =$ `0x12345678`.

Worked example 4 - ChaCha rotation by 16

$\text{rotl}_{32}(\texttt{0xA1B2C3D4}, 16) =$ `0xC3D4A1B2`. A 16-bit rotation swaps the two 16-bit halves.

Exercises

Rotate: (a) `11001001` left by 1, (b) `0x3C` left by 4, (c) `0x11223344` left by 16, (d) explain why rotation by 0 changes nothing.

6 Method 5: perform one ChaCha quarter-round

Input: a, b, c, d
Do: eight add/XOR/rotate instructions
Output: new a, b, c, d

Plain-language explanation

A quarter-round is called a quarter-round because it touches four of the sixteen state words. It is not one arithmetic operation. It is a fixed list of eight assignments. Never try to do all eight mentally.

Mechanical rule

$$
\begin{aligned}
a &\leftarrow a + b & d &\leftarrow \mathsf{rotl}(d \oplus a, 16) \\
c &\leftarrow c + d & b &\leftarrow \mathsf{rotl}(b \oplus c, 12) \\
a &\leftarrow a + b & d &\leftarrow \mathsf{rotl}(d \oplus a, 8) \\
c &\leftarrow c + d & b &\leftarrow \mathsf{rotl}(b \oplus c, 7)
\end{aligned}
$$

Every addition wraps modulo 2^{32}.

Worked example 1 - start (0x11111111, 0x01020304, 0x9B8D6F43, 0x01234567)

1. a=a+b: $(a, b, c, d) = (\texttt{0x12131415}, \texttt{0x01020304}, \texttt{0x9B8D6F43}, \texttt{0x01234567})$
2. d=rotl(d xor a,16): $(a, b, c, d) = (\texttt{0x12131415}, \texttt{0x01020304}, \texttt{0x9B8D6F43}, \texttt{0x51721330})$
3. c=c+d: $(a, b, c, d) = (\texttt{0x12131415}, \texttt{0x01020304}, \texttt{0xECFF8273}, \texttt{0x51721330})$
4. b=rotl(b xor c,12): $(a, b, c, d) = (\texttt{0x12131415}, \texttt{0xD8177EDF}, \texttt{0xECFF8273}, \texttt{0x51721330})$
5. a=a+b: $(a, b, c, d) = (\texttt{0xEA2A92F4}, \texttt{0xD8177EDF}, \texttt{0xECFF8273}, \texttt{0x51721330})$
6. d=rotl(d xor a,8): $(a, b, c, d) = (\texttt{0xEA2A92F4}, \texttt{0xD8177EDF}, \texttt{0xECFF8273}, \texttt{0x5881C4BB})$
7. c=c+d: $(a, b, c, d) = (\texttt{0xEA2A92F4}, \texttt{0xD8177EDF}, \texttt{0x4581472E}, \texttt{0x5881C4BB})$
8. b=rotl(b xor c,7): $(a, b, c, d) = (\texttt{0xEA2A92F4}, \texttt{0xCB1CF8CE}, \texttt{0x4581472E}, \texttt{0x5881C4BB})$

Final result: $\boxed{(\texttt{0xEA2A92F4}, \texttt{0xCB1CF8CE}, \texttt{0x4581472E}, \texttt{0x5881C4BB})}$.

Worked example 2 - start (0x00000000, 0x00000000, 0x00000000, 0x00000000)

1. a=a+b: $(a, b, c, d) = (\texttt{0x00000000}, \texttt{0x00000000}, \texttt{0x00000000}, \texttt{0x00000000})$
2. d=rotl(d xor a,16): $(a, b, c, d) = (\texttt{0x00000000}, \texttt{0x00000000}, \texttt{0x00000000}, \texttt{0x00000000})$
3. c=c+d: $(a, b, c, d) = (\texttt{0x00000000}, \texttt{0x00000000}, \texttt{0x00000000}, \texttt{0x00000000})$
4. b=rotl(b xor c,12): $(a, b, c, d) = (\texttt{0x00000000}, \texttt{0x00000000}, \texttt{0x00000000}, \texttt{0x00000000})$
5. a=a+b: $(a, b, c, d) = (\texttt{0x00000000}, \texttt{0x00000000}, \texttt{0x00000000}, \texttt{0x00000000})$
6. d=rotl(d xor a,8): $(a, b, c, d) = (\texttt{0x00000000}, \texttt{0x00000000}, \texttt{0x00000000}, \texttt{0x00000000})$
7. c=c+d: $(a, b, c, d) = (\texttt{0x00000000}, \texttt{0x00000000}, \texttt{0x00000000}, \texttt{0x00000000})$
8. b=rotl(b xor c,7): $(a, b, c, d) = (\texttt{0x00000000}, \texttt{0x00000000}, \texttt{0x00000000}, \texttt{0x00000000})$

Final result: $\boxed{(\texttt{0x00000000}, \texttt{0x00000000}, \texttt{0x00000000}, \texttt{0x00000000})}$.

Worked example 3 - start (0x00000001, 0x00000002, 0x00000003, 0x00000004)

1. a=a+b: $(a, b, c, d) = (\texttt{0x00000003}, \texttt{0x00000002}, \texttt{0x00000003}, \texttt{0x00000004})$
2. d=rotl(d xor a,16): $(a, b, c, d) = (\texttt{0x00000003}, \texttt{0x00000002}, \texttt{0x00000003}, \texttt{0x00070000})$
3. c=c+d: $(a, b, c, d) = (\texttt{0x00000003}, \texttt{0x00000002}, \texttt{0x00070003}, \texttt{0x00070000})$
4. b=rotl(b xor c,12): $(a, b, c, d) = (\texttt{0x00000003}, \texttt{0x70001000}, \texttt{0x00070003}, \texttt{0x00070000})$
5. a=a+b: $(a, b, c, d) = (\texttt{0x70001003}, \texttt{0x70001000}, \texttt{0x00070003}, \texttt{0x00070000})$
6. d=rotl(d xor a,8): $(a, b, c, d) = (\texttt{0x70001003}, \texttt{0x70001000}, \texttt{0x00070003}, \texttt{0x07100370})$
7. c=c+d: $(a, b, c, d) = (\texttt{0x70001003}, \texttt{0x70001000}, \texttt{0x07170373}, \texttt{0x07100370})$
8. b=rotl(b xor c,7): $(a, b, c, d) = (\texttt{0x70001003}, \texttt{0x8B89B9BB}, \texttt{0x07170373}, \texttt{0x07100370})$

Final result: $\boxed{(\texttt{0x70001003}, \texttt{0x8B89B9BB}, \texttt{0x07170373}, \texttt{0x07100370})}$.

Worked example 4 - start $(\texttt{0xFFFFFFFF}, \texttt{0x00000001}, \texttt{0x00000000}, \texttt{0x00000000})$

1. a=a+b: $(a,b,c,d) = (\texttt{0x00000000}, \texttt{0x00000001}, \texttt{0x00000000}, \texttt{0x00000000})$
2. d=rotl(d xor a,16): $(a,b,c,d) = (\texttt{0x00000000}, \texttt{0x00000001}, \texttt{0x00000000}, \texttt{0x00000000})$
3. c=c+d: $(a,b,c,d) = (\texttt{0x00000000}, \texttt{0x00000001}, \texttt{0x00000000}, \texttt{0x00000000})$
4. b=rotl(b xor c,12): $(a,b,c,d) = (\texttt{0x00000000}, \texttt{0x00001000}, \texttt{0x00000000}, \texttt{0x00000000})$
5. a=a+b: $(a,b,c,d) = (\texttt{0x00001000}, \texttt{0x00001000}, \texttt{0x00000000}, \texttt{0x00000000})$
6. d=rotl(d xor a,8): $(a,b,c,d) = (\texttt{0x00001000}, \texttt{0x00001000}, \texttt{0x00000000}, \texttt{0x00100000})$
7. c=c+d: $(a,b,c,d) = (\texttt{0x00001000}, \texttt{0x00001000}, \texttt{0x00100000}, \texttt{0x00100000})$
8. b=rotl(b xor c,7): $(a,b,c,d) = (\texttt{0x00001000}, \texttt{0x08080000}, \texttt{0x00100000}, \texttt{0x00100000})$

Final result: $\boxed{(\texttt{0x00001000}, \texttt{0x08080000}, \texttt{0x00100000}, \texttt{0x00100000})}$.

Exercises

Use a calculator that supports hexadecimal, but write all eight lines. Compute quarter-rounds for: (a) $(1,0,0,0)$, (b) $(0,1,0,0)$, (c) $(\texttt{0xFFFFFFFF}, 0, 0, 0)$, (d) verify the RFC example output shown above.

7 Method 6: choose the words for column and diagonal rounds

Input: a 4 by 4 state
Do: apply eight quarter-rounds at fixed indices
Output: one double-round result

Plain-language explanation

The state is stored as a flat array numbered 0 through 15, but drawing it as a grid makes the pattern visible.

$$\begin{bmatrix} 0 & 1 & 2 & 3 \\ 4 & 5 & 6 & 7 \\ 8 & 9 & 10 & 11 \\ 12 & 13 & 14 & 15 \end{bmatrix}$$

First mix four vertical columns. Then mix four diagonals that wrap around the edges. Together these are two rounds, often called one double-round.

Mechanical rule

Column groups: $(0, 4, 8, 12)$, $(1, 5, 9, 13)$, $(2, 6, 10, 14)$, $(3, 7, 11, 15)$.
Diagonal groups: $(0, 5, 10, 15)$, $(1, 6, 11, 12)$, $(2, 7, 8, 13)$, $(3, 4, 9, 14)$.

Worked example 1 - locate one column

The group $(2, 6, 10, 14)$ is the third visual column. Only those four boxes are passed to the quarter-round.

Worked example 2 - locate one wrapped diagonal

The group $(1, 6, 11, 12)$ moves one step right per row and wraps from column 3 back to column 0.

Worked example 3 - toy 8-bit double-round

For readability this toy uses 8-bit words and rotation distances $4, 3, 2, 1$. The index pattern is identical to real ChaCha20.

before $\begin{bmatrix} \texttt{0x00} & \texttt{0x01} & \texttt{0x02} & \texttt{0x03} \\ \texttt{0x04} & \texttt{0x05} & \texttt{0x06} & \texttt{0x07} \\ \texttt{0x08} & \texttt{0x09} & \texttt{0x0A} & \texttt{0x0B} \\ \texttt{0x0C} & \texttt{0x0D} & \texttt{0x0E} & \texttt{0x0F} \end{bmatrix}$ $\longrightarrow$ after $\begin{bmatrix} \texttt{0xF6} & \texttt{0x50} & \texttt{0xA1} & \texttt{0xFA} \\ \texttt{0x45} & \texttt{0x08} & \texttt{0x75} & \texttt{0x87} \\ \texttt{0x14} & \texttt{0xD2} & \texttt{0x03} & \texttt{0xC0} \\ \texttt{0x1C} & \texttt{0x6F} & \texttt{0x42} & \texttt{0x9C} \end{bmatrix}$

Worked example 4 - toy 8-bit double-round

For readability this toy uses 8-bit words and rotation distances $4, 3, 2, 1$. The index pattern is identical to real ChaCha20.

before $\begin{bmatrix} \texttt{0x01} & \texttt{0x02} & \texttt{0x03} & \texttt{0x04} \\ \texttt{0x05} & \texttt{0x06} & \texttt{0x07} & \texttt{0x08} \\ \texttt{0x09} & \texttt{0x0A} & \texttt{0x0B} & \texttt{0x0C} \\ \texttt{0x0D} & \texttt{0x0E} & \texttt{0x0F} & \texttt{0x10} \end{bmatrix}$ $\longrightarrow$ after $\begin{bmatrix} \texttt{0x2E} & \texttt{0x15} & \texttt{0xF7} & \texttt{0x33} \\ \texttt{0x77} & \texttt{0x1D} & \texttt{0x71} & \texttt{0xDE} \\ \texttt{0x89} & \texttt{0xC9} & \texttt{0xD6} & \texttt{0xF3} \\ \texttt{0xBD} & \texttt{0x90} & \texttt{0x00} & \texttt{0xF6} \end{bmatrix}$

Exercises

(a) Circle the four indices in the diagonal group starting at index 2. (b) Which groups contain index 12? (c) Apply the toy double-round to sixteen zero bytes. (d) Explain why no state word is left untouched after a double-round.

8 Method 7: build the real ChaCha20 input state

Input: constant, key, counter, nonce
Do: load bytes little-endian into fixed positions
Output: sixteen 32-bit words

Mechanical rule

$$\begin{bmatrix} C_0 & C_1 & C_2 & C_3 \\ K_0 & K_1 & K_2 & K_3 \\ K_4 & K_5 & K_6 & K_7 \\ \text{ctr} & N_0 & N_1 & N_2 \end{bmatrix}$$

The four constants spell “expand 32-byte k” when their bytes are read in memory order.

Worked example 1 - constants

Bytes `65 78 70 61` become `0x61707865`. The other constant words are `0x3320646E`, `0x79622D32`, `0x6B206574`.

Worked example 2 - first four key words

Key bytes `00 01 ... 0F` become $K_0 =$ `0x03020100`, $K_1 =$ `0x07060504`, $K_2 =$ `0x0B0A0908`, $K_3 =$ `0x0F0E0D0C`.

Worked example 3 - counter

Counter value 1 is stored as the word `0x00000001`. When serialized to bytes later it appears as `01 00 00 00`.

Worked example 4 - complete RFC input state

0x61707865	0x3320646E	0x79622D32	0x6B206574
0x03020100	0x07060504	0x0B0A0908	0x0F0E0D0C
0x13121110	0x17161514	0x1B1A1918	0x1F1E1D1C
0x00000001	0x09000000	0x4A000000	0x00000000

Exercises

Build the final row for: (a) counter 5 and nonce bytes `00...00`; (b) counter `0x01020304` and nonce `01 02 03 04 05 06 07 08 09 0A 0B 0C`; (c) explain which state words change between block 7 and block 8; (d) explain which words are secret.

9 Method 8: produce one 64-byte ChaCha20 block

Input:	initial 16-word state
Do:	copy, mix for 20 rounds, add original, serialize
Output:	64 keystream bytes

Plain-language explanation

There are two states during the block function. The **initial state** must be kept unchanged. The **working state** is mixed. After 20 rounds, add the initial state back word by word. This last addition is called feed-forward.

Mechanical rule

1. $W \leftarrow S$.
2. Repeat the eight quarter-round schedule ten times. This gives 20 rounds.
3. For every i, set $W_i \leftarrow W_i + S_i \pmod{2^{32}}$.
4. Store each word as four little-endian bytes.

Worked example 1 - why a copy is required

If the initial word is `0x00000009` and the mixed word is `0xFFFFFFFE`, feed-forward gives `0x00000007`. Without the saved initial word this result cannot be formed.

Worked example 2 - serialize one final word

Final word `0xE4E7F110` becomes bytes `10 F1 E7 E4`. The visual order reverses by byte groups.

Worked example 3 - output length

Sixteen words times four bytes per word gives $16 \cdot 4 = 64$ output bytes, even when the plaintext is shorter.

Worked example 4 - RFC block prefix

For the RFC key, nonce, and counter 1, the block starts `10 F1 E7 E4 D1 3B 59 15 50 0F DD 1F A3 20 71 C4`. Tyr-Crypto embeds this test vector in `chacha20.nim`.

Exercises

(a) Serialize `0x12345678`. (b) How many quarter-round calls occur in one block? (c) How many bytes do three blocks provide? (d) Why must the original state not be modified before feed-forward?

10 Method 9: encrypt and decrypt by XORing a keystream

Input:	message bytes and keystream bytes
Do:	XOR matching positions
Output:	ciphertext or recovered plaintext

Mechanical rule

$C_i = P_i \oplus Z_i$ and $P_i = C_i \oplus Z_i$. The same function can encrypt and decrypt because XORing the same keystream twice cancels it.

Worked example 1 - one byte

$P = \texttt{0x41}$ and $Z = \texttt{0xE3}$ give $C = \texttt{0xA2}$. Then $\texttt{0xA2} \oplus \texttt{0xE3} = \texttt{0x41}$.

Worked example 2 - four bytes

Plaintext `01 02 03 04` XOR keystream `10 20 30 40` gives ciphertext `11 22 33 44`.

Worked example 3 - partial final block

A 70-byte message uses 64 bytes from block c and only the first 6 bytes from block $c + 1$. The unused 58 bytes are discarded.

Worked example 4 - empty input

An empty message produces an empty ciphertext. Tyr-Crypto's range check returns before requesting any block.

Exercises

(a) XOR `AA BB CC` with `01 02 03`. (b) Recover plaintext from `FE DC` with keystream `11 22`. (c) How many blocks for 129 bytes? (d) Which keystream positions encrypt bytes 64 through 69?

11 Method 10: advance and protect the block counter

Input:	initial counter and message length
Do:	count required 64-byte blocks
Output:	safe counter interval

Mechanical rule

For positive length L:

$$\text{blocks} = \left\lfloor \frac{L-1}{64} \right\rfloor + 1.$$

The last counter used is $c + \text{blocks} - 1$. It must not exceed `0xFFFFFFFF`.

Worked example 1 - 1 byte

$L = 1$ needs one block. Starting at counter 7 uses only counter 7.

Worked example 2 - exactly 64 bytes

$L = 64$ also needs one block. The next call may start at counter $c + 1$ if the same key and nonce are intentionally continued.

Worked example 3 - 65 bytes

$L = 65$ needs two blocks: counters c and $c + 1$.

Worked example 4 - rejected wrap

Starting at `0xFFFFFFFF`, a 65-byte message would require counters `0xFFFFFFFF` and `0x00000000`. Tyr-Crypto rejects this instead of wrapping.

Exercises

Find block counts for 0, 63, 64, 128, and 129 bytes. Then decide whether 128 bytes are allowed from initial counter `0xFFFFFFFE`.

12 Method 11: derive an HChaCha20 subkey

Input: 32-byte key and first 16 nonce bytes
Do: ChaCha rounds without counter and feed-forward
Output: 32-byte subkey

Plain-language explanation

HChaCha20 reuses the ChaCha mixing core, but its purpose is different. It does not output a keystream block. It creates a new key. The last state row contains four nonce words instead of counter plus three nonce words.

Mechanical rule

Build $[C_0..C_3 \mid K_0..K_7 \mid N_0..N_3]$, run 20 rounds, do **not** add the initial state, then output words 0–3 and 12–15.

Worked example 1 - state placement

The 16-byte nonce `00 00 00 09 00 00 00 4A 00 00 00 00 31 41 59 27` becomes words `0x09000000`, `0x4A000000`, `0x00000000`, `0x27594131`.

Worked example 2 - extraction pattern

If the final rows are (A, B, C, D) and (M, N, O, P), the subkey is the little-endian serialization of A, B, C, D, M, N, O, P. Middle words 4–11 are ignored.

Worked example 3 - official final state

The draft test vector ends with first row `0x423B4182 0xFE7BB227 0x50420ED3 0x737D878A` and last row `0xD5E4F9A0 0x53A8748A 0x13C42EC1 0xDCECD326`.

Worked example 4 - serialized subkey prefix

Word `0x423B4182` serializes as `82 41 3B 42`; therefore the subkey begins `82 41 3B 42 27 B2 7B FE ....`

Exercises

(a) Which eight final indices are returned? (b) Why is there no feed-forward? (c) Serialize `0xD5E4F9A0`. (d) State the nonce length accepted by Tyr-Crypto's HChaCha function.

13 Method 12: construct XChaCha20

Input: 32-byte key, 24-byte nonce, message
Do: derive subkey, rebuild 12-byte nonce, run ChaCha20
Output: ciphertext or plaintext

Mechanical rule

1. $K' \leftarrow \text{HChaCha20}(K, N[0..15])$.
2. Build $N' = \texttt{00 00 00 00} \,\|\, N[16..23]$.
3. Run ordinary ChaCha20 with K', N', and the chosen counter.

Worked example 1 - split the nonce

For nonce bytes numbered 0 through 23, bytes 0–15 derive the subkey. Bytes 16–23 remain visible in the final ChaCha nonce.

Worked example 2 - construct the 12-byte nonce

Remaining bytes `A0 A1 A2 A3 A4 A5 A6 A7` become `00 00 00 00 A0 A1 A2 A3 A4 A5 A6 A7`.

Worked example 3 - plain stream mode

Tyr-Crypto's convenience overload starts the counter at 0. This is appropriate for bare stream generation, but it is not the AEAD counter allocation explained in Volume 2.

Worked example 4 - why the nonce is longer

A 24-byte nonce has 192 bits. Its first 128 bits select a subkey; its last 64 bits select the ordinary ChaCha stream under that subkey.

Do not skip this warning

A longer nonce reduces accidental collision risk, but nonce reuse with the same key is still not a feature. Furthermore, bare XChaCha20 does not authenticate data.

Exercises

(a) Split a 24-byte nonce into its two roles. (b) Construct the final nonce from trailing bytes `01 02 03 04 05 06 07 08`. (c) Which key is passed to ordinary ChaCha20? (d) Which counter should an AEAD composition reserve for one-time Poly1305 key generation?

14 Reading the Tyr-Crypto implementation

Plain-language explanation

The code follows the calculation order closely. Reading it is easier when each line is attached to the method number learned above.

```
proc quarterRound(a, b, c, d: var uint32) =
  a = a + b
  d = rotateLeftBits(d xor a, 16)
  c = c + d
  b = rotateLeftBits(b xor c, 12)
  a = a + b
  d = rotateLeftBits(d xor a, 8)
  c = c + d
  b = rotateLeftBits(b xor c, 7)
```

Source item	Calculation meaning
`load32Le`	Method 1: four bytes to one little-endian word.
`quarterRound`	Methods 2–5: wrapping add, XOR, rotations.
`writeChaCha20Block`	Methods 6–8: rounds, feed-forward, serialization.
`chacha20XorInPlace`	Methods 9–10: block loop and XOR into an existing buffer.
`hchacha20`	Method 11: subkey derivation and secret-state clearing.
`xchacha20Xor`	Method 12: nonce split, subkey, ordinary ChaCha call.

Do not skip this warning

The code uses `defer` and explicit secure-clear helpers for temporary key material and states. This reduces obvious lifetime problems, but compiler behavior and platform behavior still matter. A source-level wipe is not a formal guarantee against every microarchitectural leak.

15 Cumulative exercises

Cumulative A - one complete quarter-round

Start with $a =$ `0x00000001`, $b =$ `0x00000002`, $c =$ `0x00000003`, $d =$ `0x00000004`. Copy the eight-line recipe and show every intermediate tuple.

Cumulative B - state and block planning

A 150-byte message uses key bytes `00...1F`, a 12-byte nonce `00...0B`, and initial counter 9. Build the complete initial state for the first block, state the counters used, and identify how many bytes are taken from the final block.

Cumulative C - XChaCha construction

Given a 24-byte nonce `00 01 ... 17`, mark the bytes sent to HChaCha20, write the final 12-byte ChaCha nonce, and explain which intermediate value must be wiped.

Cumulative D - misuse diagnosis

A program encrypts two different messages with the same key, nonce, and counter. Show algebraically that $C_1 \oplus C_2 = P_1 \oplus P_2$. State what this reveals and why adding Poly1305 after the fact does not repair already reused keystream.

16 Solutions

Methods 1–4: words and bit operations

Method 1. The first visible byte receives weight 2^0, the next 2^8, and so on: (a) `0xEFBEADDE`; (b) `0x13121110`; (c) `0x004A0000`; (d) `0x61707865`.
Method 2. Add normally and discard every bit above bit 31: (a) `0x00000010`; (b) `0x00000001`; (c) `0x00000100`; (d) `0x80000000`.
Method 3. Compare corresponding bits: (a) `0x5A`; (b) `0xFFFFFFFF`; (c) `0x00000000`; (d) $P = C \oplus K =$ `0x93` $\oplus$ `0xA5` $=$ `0x36`.
Method 4. (a) `10010011`; (b) `0xC3`; (c) `0x33441122`; (d) a rotation by zero moves every bit back to its original position.

Methods 5–8: quarter-round, state, and block

Method 5. Apply the eight assignments in their printed order. Final tuples are:
(a) $($`0x10000001`$,$ `0x80808808`$,$ `0x01010110`$,$ `0x01000110`$)$;
(b) $($`0x10001001`$,$ `0x88888808`$,$ `0x01110110`$,$ `0x01100110`$)$;
(c) $($`0xFFFFFFFE`$,$ `0xFFFF807F`$,$ `0x000000FF`$,$ `0x00000100`$)$;
(d) $($`0xEA2A92F4`$,$ `0xCB1CF8CE`$,$ `0x4581472E`$,$ `0x5881C4BB`$)$. A mismatch means one of the previous seven intermediate tuples must be checked before the final line.
Method 6. (a) The diagonal beginning at index 2 is $(2, 7, 8, 13)$. (b) Index 12 occurs in column $(0, 4, 8, 12)$ and diagonal $(1, 6, 11, 12)$. (c) Sixteen zero toy words stay zero because add, XOR, and rotation all preserve zero. (d) The four column groups partition all sixteen indices, and the four diagonal groups partition them a second time.
Method 7. (a) Final row: $($`0x00000005`$, 0, 0, 0)$.
(b) Final row: $($`0x01020304`$,$ `0x04030201`$,$ `0x08070605`$,$ `0x0C0B0A09`$)$.
(c) Only word 12, the counter word, changes. (d) Words 4–11 contain the secret key; constants, counter, and nonce are normally public.
Method 8. (a) `0x12345678` becomes `78 56 34 12`. (b) One block uses $8 \times 10 = 80$ quarter-round calls. (c) Three blocks provide 192 bytes. (d) Feed-forward must add the untouched initial state; overwriting it would add the mixed state to itself and compute a different function.

Methods 9–12: stream processing and XChaCha20

Method 9. (a) `AB B9 CF`; (b) `EF FE`; (c) $\lceil 129/64 \rceil = 3$ blocks; (d) message bytes 64–69 use keystream positions 0–5 of the second block.
Method 10. Lengths 0, 63, 64, 128, 129 require 0, 1, 1, 2, 3 blocks. From `0xFFFFFFFE`, 128 bytes use `0xFFFFFFFE` and `0xFFFFFFFF`; this fits exactly and does not wrap.
Method 11. (a) Return indices 0,1,2,3,12,13,14,15. (b) Feed-forward is omitted because HChaCha20 exports a derived key, not a ChaCha keystream block. (c) `0xD5E4F9A0` serializes to `A0 F9 E4 D5`. (d) HChaCha20 accepts 16 nonce bytes.
Method 12. (a) Nonce bytes 0–15 go to HChaCha20; bytes 16–23 become the visible tail of the final nonce. (b) `00 00 00 00 01 02 03 04 05 06 07 08`. (c) Ordinary ChaCha20 receives the 32-byte HChaCha20 subkey. (d) A ChaCha20-Poly1305 AEAD composition reserves counter 0 for its one-time MAC key and begins plaintext at counter 1; bare Tyr XChaCha20 may instead start at counter 0.

Cumulative solution A: all eight quarter-round lines

Starting tuple: $(1, 2, 3, 4)$.
1. $($`0x00000003`$,$ `0x00000002`$,$ `0x00000003`$,$ `0x00000004`$)$
2. $($`0x00000003`$,$ `0x00000002`$,$ `0x00000003`$,$ `0x00070000`$)$
3. $($`0x00000003`$,$ `0x00000002`$,$ `0x00070003`$,$ `0x00070000`$)$
4. $($`0x00000003`$,$ `0x70001000`$,$ `0x00070003`$,$ `0x00070000`$)$
5. $($`0x70001003`$,$ `0x70001000`$,$ `0x00070003`$,$ `0x00070000`$)$
6. $($`0x70001003`$,$ `0x70001000`$,$ `0x00070003`$,$ `0x07100370`$)$
7. $($`0x70001003`$,$ `0x70001000`$,$ `0x07170373`$,$ `0x07100370`$)$
8. $\boxed{($`0x70001003`$,$ `0x8B89B9BB`$,$ `0x07170373`$,$ `0x07100370`$)}$.

Cumulative solutions B–D

B. The first block state is
`61707865 3320646E 79622D32 6B206574`
`03020100 07060504 0B0A0908 0F0E0D0C`
`13121110 17161514 1B1A1918 1F1E1D1C`

`00000009 03020100 07060504 0B0A0908`.
There are three blocks with counters 9, 10, and 11. The final block contributes $150 - 128 = 22$ bytes.
C. Bytes `00..0F` enter HChaCha20. The final nonce is `00 00 00 00 10 11 12 13 14 15 16 17`. Wipe the derived 32-byte subkey as soon as the ordinary ChaCha call no longer needs it.
D. $(P_1 \oplus Z) \oplus (P_2 \oplus Z) = P_1 \oplus P_2$ because $Z \oplus Z = 0$. This exposes the relation between plaintexts and often allows guessed text to recover both. A later tag cannot undo ciphertexts already produced with a repeated stream.

17 References and code map

Source	Link
Tyr ChaCha20 source	https://github.com/siriuslee69/Tyr-Crypto/blob/fb9cf27926140e95c02362d4fe1c349ff65d3dd0/src/protocols/custom_crypto/symmetric/chacha/chacha20.nim
Tyr XChaCha20 source	https://github.com/siriuslee69/Tyr-Crypto/blob/fb9cf27926140e95c02362d4fe1c349ff65d3dd0/src/protocols/custom_crypto/symmetric/chacha/xchacha20.nim
RFC 8439	https://www.rfc-editor.org/rfc/rfc8439
XChaCha draft	https://datatracker.ietf.org/doc/html/draft-irtf-cfrg-xchacha-03

Do not skip this warning

This handbook explains calculations and repository structure. It is not a security certification. Do not deploy custom cryptographic code solely because worked examples match.